

IODA-ARCHITEKTUR, TEIL 3

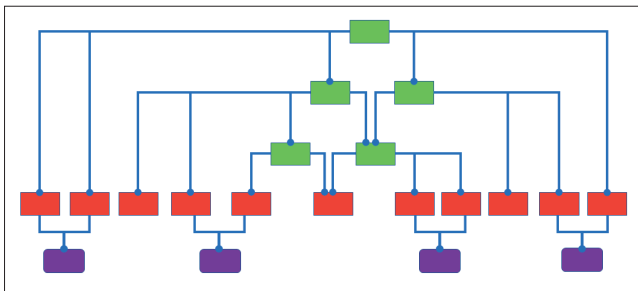
IODA am Beispiel

Die Wortzählung veranschaulicht die Anwendung der IODA-Architektur in der Praxis.

Nachdem die beiden vorangegangenen Artikel [1] und [2] einen so langen Ausflug in die Theorie unternommen haben, kehren wir nun zurück zur Praxis. Sie fragen sich sicher, wie denn eine Architektur ohne funktionale Abhängigkeiten überhaupt zu einem lauffähigen Programm führen kann.

In **Bild 1** sehen Sie deshalb die bereits bekannte Wortzählung in einer IODA-Architektur. Kontrastierend zur Clean Architecture ist darin vor allem Folgendes hervorhebenswert:

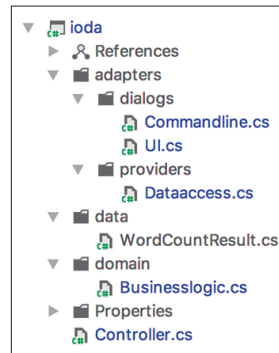
- Die IODA-Architektur betont die Notwendigkeit der Kapselung von APIs in Portalen und Providern. Deshalb habe ich hier erstmals den Zugriff auf den Kommandozeilenparameter in einen eigenen Dialog herausgezogen: die Klasse *CommandLine*; sie verbirgt den Umgang mit dem „API“ *string[] args*. Über die Kommandozeile teilt der Benutzer der Anwendung etwas mit. Demgegenüber ist das UI für die Kommunikation mit dem Benutzer per Standard-Input-/Output zuständig.
- In einer IODA-Architektur sind die Operationen durch gemeinsame Daten verbunden (**Bild 2**). Sie legt damit nahe, sich darüber Gedanken zu machen und insbesondere die fließenden Daten durch eigene Datentypen auszudrücken. Das habe ich einmal exemplarisch mit *WordCountResult* getan (vergleiche **Bild 3**).



Operationen stehen über Daten in Kontakt (**Bild 2**)

- Der Eintritt in Anwendungen geschieht immer über Controller. Im Beispiel gibt es nur einen Entry Point, der mit einer gleichnamigen Klasse hervorgehoben ist. In bisherigen Architekturmodellen sind Entry Points kein Thema.

Vor allem erwähnenswert ist jedoch das, was fehlt. Haben Sie es bemerkt? In **Bild 1** gibt es keine Interfaces! Der Code gemäß IODA-Architektur leidet nicht unter der zusätzlichen Komplexität, die das Dependency Inversion Principle (DIP) erzeugt. Er ist damit schon auf dieser Ebene viel verständlicher, wie ich finde. Er ist weniger verrauscht. Sie müssen



Das Wortzählungs-Beispiel, strukturiert nach dem IODA-Architekturmuster (**Bild 1**)

nicht überlegen, wie Abhängigkeiten zur Compilezeit versus Laufzeit verlaufen.

Das soll nicht bedeuten, dass das DIP keinen Platz in einer Anwendung nach IODA-Architektur hat. Obwohl es keine funktionalen Abhängigkeiten gibt, können auch dort manche Tests von einer derartigen Entkopplung profitieren – und zwar Tests von Integrationen.

In diesem Beispiel habe ich auf Interfaces aber ganz bewusst verzichtet, um den grundsätzlich anderen Ansatz der

IODA-Architektur zu unterstreichen.

Allerdings kommt das Prinzip Inversion of Control (IoC) zum Einsatz. Bei Adaptern ist es sehr wahrscheinlich, dass sie einmal (zum Beispiel in Tests) ersetzt werden müssen und also ein Interface nützlich wäre. In Zukunft. Vielleicht. Deshalb sind Adapter auch keine statischen Klassen und veröffentlichen keine statischen Methoden. Und deshalb injiziere ich Instanzen von Adaptern in die Integration. Damit sind Vorbereitungen für einen späteren Einsatz des DIP getroffen, ohne den Code dadurch allzu sehr zu verrauschen. Relevant ist das ohnehin nur für die Integration, also im Controller (**Bild 4**).

Die Abhängigkeit des Controllers von einem API besteht lediglich in der statischen Methode *Main()* mit dem *args*-Parameter. Die Integrationsleistung, die dieser Entry Point erbringt, besteht im Aufruf von *Run()*. Diese Funktion repräsentiert auf höchster funktionaler Abstraktionsebene das gesamte Verhalten der Anwendung. Konkret genutzt wird das *args*-API ohnehin erst im Dialog *CommandLine*.

Main() selbst muss nicht automatisiert getestet werden. Aber wenn ich das Anwendungsverhalten automatisiert testen wollte, würde es mir über die eine Funktion *Run()* zur Verfügung stehen. In einem Test müsste ich den Konstruktor des Controllers dann nur passend befüllen. Auch ohne Attrappen ist diese Möglichkeit nicht zu verachten. Aber wenn ich zum Beispiel in einem Test keine Benutzerinteraktion über ein UI haben möchte, könnte ich nachträglich und unmittelbar bedarfsgetrieben ohne großen Aufwand ein Interface *IUI* einführen. Die IODA-Architektur erlaubt mir, mit den Anforderungen zu wachsen. Sie dekretiert nicht von vornherein, dass und wo ich Interface-Abstraktionen einziehen muss. Komplexität durch kategoriale Abstraktionen ist nicht gesetzt und nicht modellimmanent, sondern richtet sich nach dem Bedarf.

```

class Businesslogic {
    public static WordCountResult Count_words(string text, HashSet<string> stopwordsDirectory) {
        var t = Shred(text);
        return new WordCountResult {
            CountTotal = Filter(t.Words, stopwordsDirectory).ToArray().Length,
            CountDistinct = Filter(t.UniqueWords, stopwordsDirectory).ToArray().Length
        };
    }

    private static ShredderedText Shred(string text) {
        var wordCandidates = text.Split(new[] { ' ', '\t', '\n', '\r' }, StringSplitOptions.RemoveEmptyEntries);
        return new ShredderedText(wordCandidates.SelectMany(Normalize));
    }

    IEnumerable<string> Normalize(string wordCandidate) {
        foreach (var m in Regex.Matches(wordCandidate, @"[\w-]*"))
            if (!string.IsNullOrEmpty(m.ToString()))
                yield return m.ToString();
    }

    private static IEnumerable<string> Filter(IEnumerable<string> words, HashSet<string> stopwordsDirectory)
        => words.Where(w => !stopwordsDirectory.Contains(w));
}

class ShredderedText {
    private readonly IEnumerable<string> _words;

    public ShredderedText(IEnumerable<string> words) { _words = words; }

    public string[] Words => _words.ToArray();
    public string[] UniqueWords => _words.Distinct().ToArray();
}

```

Eine Datenklasse verbindet Operationen (Bild 3)

Ein weiterer Unterschied der Implementation nach dem IODA-Architekturmodell zeigt sich, wenn ich in die Klassen hineinzoomte. Da ist erstens der Controller, der gar keine Logik enthält (Bild 5). Er ist abgesehen von *Main()* ausschließlich für die Integration zuständig. Er ist gewissermaßen eine Abstraktionsmaschine. Seine Methoden komponieren aus Teilen schrittweise etwas Umfassenderes, Abstrakteres – das an der Spitze in *Run()* kulminiert.

Wie gesagt: Wenn das Problem umfänglicher wäre, hätte ich diese Menge an Integration in eine eigene Klasse ausgelagert; doch für dieses Beispiel finde ich die Co-Lokation mit dem Entry Point in Ordnung.

Dass ich auf diese Art und Weise mit der Integration umgehe, ist andererseits auch wieder dem IODA-Architekturmodell geschuldet. Darin werden auf der grundlegenden Ebene (Bild 6) die Funktionen betont. Sie stellen das Verhalten her. Sie können IODA gewissermaßen als Function-first-Ansatz verstehen, bei dem die Abstraktion durch Komposition im Vordergrund steht.

Wo die Funktionen in Klassen zusammengefasst werden (Abstraktion durch Kategorisierung), ist dann natürlich nicht

```

internal class Controller
{
    public static void Main(string[] args) {
        var cmd = new CommandLine(args);
        var da = new Dataaccess();
        var ui = new UI();
        var app = new Controller(cmd, ui, da);

        app.Run(args);
    }

    private readonly CommandLine _cmd;
    private readonly UI _ui;
    private readonly Dataaccess _da;

    Controller(Commandline cmd, UI ui, Dataaccess da) {
        _cmd = cmd;
        _ui = ui;
        _da = da;
    }
}

```

Mit IoC wird die Integration behutsam auf einen späteren eventuellen Einsatz von DIP vorbereitet (Bild 4)

```

internal class Controller
{
    public static void Main(string[] args) {...}

    Ctor

    void Run(string[] args) {
        _cmd.Determine_text_source(
            onFile: Count_words_in_file,
            onUser: Count_words_in_user_entered_text);
    }

    void Count_words_in_file(string filename) {
        var text = _da.Load_text(filename);
        Count_words(text);
    }

    void Count_words_in_user_entered_text() {
        _ui.Ask_user_for_text(
            Count_words);
    }

    void Count_words(string text) {
        var stopwords = _da.Load_stopwords();
        var result = Businesslogic.Count_words(text, stopwords);
        _ui.Display_results(result.CountTotal, result.CountDistinct);
    }
}

```

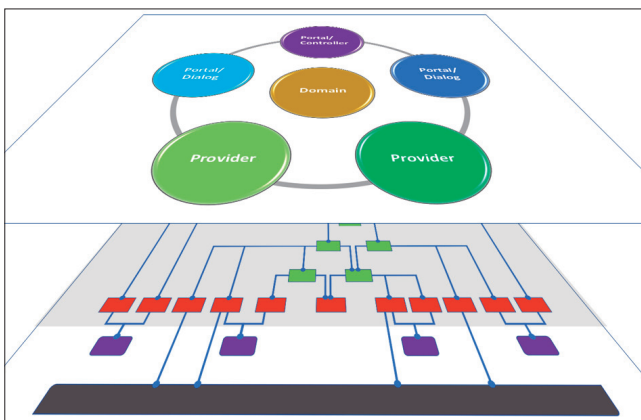
Die Leistung des Controllers besteht vor allem in der Integration (Bild 5)

egal, aber zweitrangig. Sollte die Anwendung wachsen, kann ich die integrierenden Methoden aus dem Controller leicht in eine andere Klasse verschieben.

Um die Integration allerdings auch in Entscheidungsfällen logikfrei zu halten, ist ein Trick nötig. Bild 5 zeigt in hervorgehobenen Passagen, wo ich diesen Trick angewandt habe.

Im ersten Fall geht es um die Entscheidung, ob ein Text aus einer Datei zu verarbeiten ist oder ob der Benutzer ihn eingibt. Eine solche Entscheidung erfordert Logik: eine Transformation als Bedingung plus eine Kontrollanweisung. Damit darf eine Integration nicht „kontaminiert“ werden. Solche Logik muss in einer Operation stecken, um testbar zu sein.

Um dennoch ein alternatives Verhalten der Integration zu erhalten, wird das Ergebnis der Entscheidung über Continuations (Funktionszeiger) aus der Operation herausgereicht (Bild 7). Je nachdem, ob die Verarbeitung einer Datei (*onFile*) oder einer Benutzereingabe nötig ist (*onUser*), wird die eine oder andere Methode aufgerufen. Das ist für Sie sicher ungewohnt – aber es dient dem Grundsatz der IODA-Architektur: Funktionale Abhängigkeiten sind zu vermeiden! ►



Die zwei Ebenen der IODA-Architektur (Bild 6)

```

class CommandLine
{
    private readonly string[] _args;

    public CommandLine(string[] args) { _args = args; }

    public void Determine_text_source(Action<string> onFile, Action onUser) {
        if (_args.Length > 0)
            onFile(_args[0]);
        else
            onUser();
    }
}

```

Das Ergebnis einer Entscheidung gibt eine Operation über Continuations bekannt (Bild 7)

Das Prinzip hat also Vorrang vor der Implementation. Das ist nicht anders als in der Clean Architecture. Dort führt es zu einem Anstieg der Komplexität durch Interfaces. Hier führt es in manchen Fällen lediglich zur Nutzung des ungewohnten programmiersprachlichen Mittels Funktionszeiger.

Bitte lassen Sie hier jedoch aus einem Stirnrunzeln wegen mangelnder Gewohnheit keine Abneigung gegen die IODA-Architektur entstehen. Seien Sie versichert, dass es zu solchen Konstrukten nicht in allen Entscheidungsfällen kommen muss. Ich habe es hier lediglich zur Unterstreichung des Prinzips gewählt.

Es ist eine Sache der Übung, Code wie den aus Bild 5 zu lesen. Wenn Sie das ein paarmal getan haben, fällt es Ihnen leicht. Das ist zumindest meine Erfahrung nach Jahren, in denen ich so gearbeitet und es in Clean-Code-Development-Trainings vermittelt habe. Auch DIP und IoC waren am Anfang ungewohnt in ihrer Manifestation im Code. Heute gehen wir damit routiniert um – merken aber eben leider nicht mehr, welcher Overhead dadurch entstehen kann.

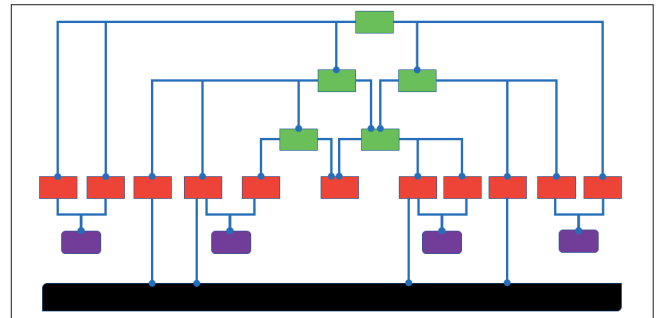
Ebenso ist es mit Continuations als Mittel, um Logik auf Operationen zu beschränken. Wenn Sie heute damit anfangen, ist es morgen Routine. Achtung: Das ist allerdings kein Selbstzweck! Wichtiger als die Einhaltung des Prinzips ist immer noch der eigentliche Zweck, nämlich Wandelbarkeit des Codes durch Verständlichkeit und Testbarkeit zu erreichen. Mehr dazu finden Sie in meinem Blog-Artikel unter [3].

Falls ein Spritzer Logik in einer Integration weder die Verständlichkeit noch die Testbarkeit spürbar beeinträchtigt, ist die Logik dort verzeihlich. Doch das ist mir für die hiesige Darstellung eigentlich schon zu differenziert. Deshalb habe ich in der Beispielimplementation darauf verzichtet.

Struktur fraktal

Die Funktionshierarchie der IODA-Umsetzung folgt wieder der Hierarchie aus Bild 8: integrierende Funktionen über Operationen über Daten (Bild 9). Funktionen in grünen Kästen enthalten keine Logik und haben ausgehende Pfeile, rufen also andere Funktionen auf. Funktionen in roten Kästen sind die Blätter des Aufrufbaumes, enthalten Logik, haben aber keine ausgehenden Pfeile. So einfach wie konsequent.

Das Ergebnis ist extrem gute Testbarkeit, wo sie gebraucht wird: dort, wo Logik steht, bei den Operationen. Außerdem erhält man hohe Verständlichkeit durch die Einhaltung des SLA in den Integrationen (vergleiche Bild 5).



Nur Operationen rufen externe APIs auf (Bild 8)

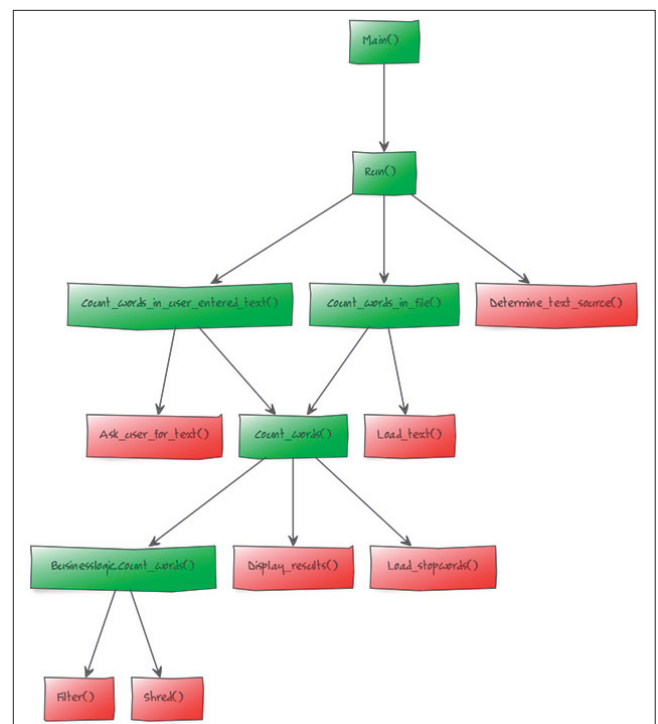
Doch nicht nur die Funktionshierarchie folgt dem IODA-Modell. Auch die Klassen sind so strukturiert (Bild 10). Es gibt eine Klasse mit der Verantwortlichkeit Integration, mehrere mit der Verantwortlichkeit Operation und auch eine Datenklasse.

Wenn Sie genau hinschauen, dann fällt Ihnen jetzt allerdings vielleicht eine Diskrepanz auf. Die Klasse *Businesslogic* ist ein Blatt im Klassendiagramm und also als operational eingestuft. Im Funktionsbaum sind hingegen nicht alle ihre Methoden als Operationen eingetragen. Ihr *Count_words()* ist eine Integration (vergleiche Bild 3). Wie kann das sein?

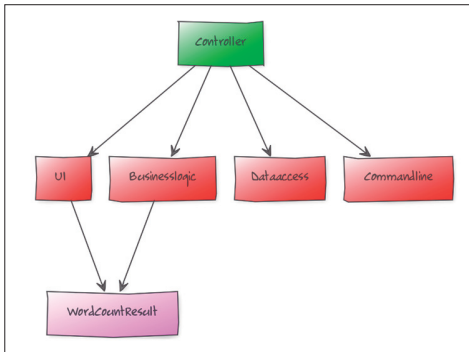
Die Lösung: Das IODA-Architekturmodell ist fraktal. Es kann eine IODA-Hierarchie innerhalb einer IODA-Hierarchie existieren (Bild 11).

Die Hierarchie aus Integration, Operation, Daten und APIs ist zwar zunächst auf Funktionen gemünzt. Dort gilt es, aus der funktionalen Abhängigkeit hinauszukommen.

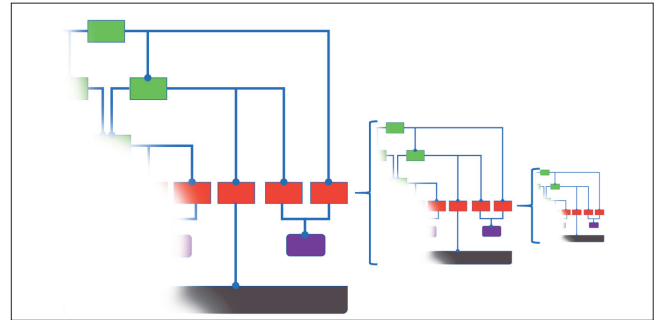
Wenn Funktionen jedoch mit Modulen kategorisiert, das heißt nach dem Zweck zusammengefasst werden, dann erge-



Der Funktionsbaum, nach IODA strukturiert (Bild 9)



Auch Klassen können dem IODA-Schema folgen (Bild 10)



Das IODA-Architekturmodell ist fraktal (Bild 11)

ben sich durch die Beziehungen zwischen den Funktionen auch Beziehungen zwischen den Modulen. Und Module bekommen einen Platz in einer Hierarchie, der von ihren Funktionen abhängt.

Module, deren Integrationen keine Funktionen außerhalb des Moduls aufrufen, sind in der Modulhierarchie dann Blätter. *Businesslogic* ist dafür ein Beispiel. Intern jedoch können auch solche Module aus Integrationen und Operationen bestehen.

Falls Module allerdings Integrationen enthalten, die Funktionen aus anderen nutzen, sind sie Knoten in der Modulhierarchie und werden als integrierende Module gewertet – selbst wenn sie Operationen enthalten sollten.

Ganz formal betrachtet mag dies ein Widerspruch sein. Darf ein Modul eine Integration sein, auch wenn es darin Operationen gibt? Wenn Sie wollen, können Sie solche Module in einer Hierarchie kennzeichnen. Ich tue das allerdings nicht. Wichtiger scheint mir, Augenmaß zu bewahren und zu schauen, wie der Anteil von Integration zu Operation in solch einem Hybriden ist. Worum geht es bei dem Modul? Ist der Zweck integrativ oder operativ?

Wenn der Schwerpunkt bei integrierenden Funktionen liegt und der Zweck integrativ ist, dann besteht kein Anlass zur Sorge. Wenn das jedoch nicht der Fall ist, dann kann genau dieser hybride Charakter der Anlass sein, ein solches Modul zu refaktorisieren und die wirklich integrierenden Teile von den operierenden zu trennen.

Und noch eines ist bei der Klasse *Businesslogic* hervorzuheben. Schauen Sie noch einmal bei Bild 3 vorbei. Fällt Ihnen bei den Funktionen etwas auf? Sie sind alle statisch.

Ich habe die Domänenlogik statisch gemacht, weil es schlicht keinen Grund gibt, es nicht zu tun. In einer IODA-Architektur ist das völlig in Ordnung, ja sogar naheliegend. Hier werden keine Interfaces zwischen allen möglichen Klassen gebraucht. Warum sollte ich also die Fachlogik mit einem Interface ausstatten oder auch nur mit Instanzmethoden?

Instanzmethoden sind schlicht weniger testbar als statische Methoden. Das vermeide ich, wo es geht. Und da es bei der Fachlogik kein Signal für Instanzmethoden gibt – das wären zum Beispiel ein gemeinsamer Zustand oder API-Aufrufe –, reichen mir statische Methoden völlig aus.

Das ist kein blinder Rückfall in ein Zeitalter prozeduraler Programmierung, sondern eine bewusste Entscheidung. Sie folgt Prinzipien und einer Systematik. Den Rahmen gibt die

IODA-Architektur vor. Den spannt sie auf, um Verständlichkeit und Testbarkeit zu motivieren.

Zusammenfassung

Die Entwicklung der Architekturmuster von MVC über das Schichtenmodell bis zur Clean Architecture ist konsequent – aber ich halte sie nicht für erschöpfend. Es kann und muss weitergehen. Einiges ist schon besser geworden, doch zu viel liegt mit der Sauberkeit von Code noch im Argen, selbst wenn diese Architekturmodelle tapfer angewandt werden.

Das zentrale Problem der funktionalen Abhängigkeiten ist in ihnen nicht wirklich gelöst. Lediglich DIP/IOC hat Linderung gebracht – und das zu einem hohen Preis: Komplexität wurde mit Komplexität bekämpft.

Mit der IODA-Architektur habe ich Ihnen nun eine Alternative vorgestellt, die das Problem an der Wurzel packt. Die Vorteile der Architekturmuster werden beibehalten, der zentrale Nachteil verschwindet. Unerwartet entstehen dabei zwei positive Effekte: Daten werden plötzlich im Modell sichtbar und Kompositionsabstraktionen entstehen ganz natürlich.

Nach all den Jahren in tiefen funktionalen Abhängigkeiten ist das IODA-Denken natürlich eine gewaltige Umstellung. Doch die lohnt sich. Und irgendwann fragen Sie sich dann, warum die Strukturierung von Software Ihnen einmal schwergefallen ist und das Testen Ihnen solche Mühe bereitet hat. ■

- [1] Ralf Westphal, *Eine Kritik bisheriger Architekturmodelle*, dotnetpro 8/2018, Seite 24 ff., www.dotnetpro.de/A1808IODA
- [2] Ralf Westphal, *Das IODA-Architekturmodell*, dotnetpro 9/2018, Seite 28 ff., www.dotnetpro.de/A1809IODA
- [3] Ralf Westphal, *Kontrollstrukturen in der Integration*, www.dotnetpro.de/SL1810IODA1



Ralf Westphal

Ralf Westphal ist Trainer, Berater, Autor und Referent auf Entwicklerkonferenzen sowie Mitgründer von Clean Code Developer. Seine Schwerpunkte sind nachhaltige Softwarearchitektur und zukunftsfähige Teamorganisation.
info@ralfw.de